RespFish

Tech Implementation

Recording respiration data

To read the (analog) respiration belt signal from the ADI device, we replace old Arduino with faster ESP - for example **ESP WROOM 02D** (https://www.reichelt.at/at/de/shop/produkt/entwicklungsboard_esp-wroom-02d-311743). It passes the data to Unity via serial (USB) or over a local network (UDP).
For the mobile phone app we will use the microphone (of mobile phone headphones) as a breath sensor.

Tech Stack

Layer 1: Acquisition
The breath recoding can be done in any programming language and on any device. The only requirement is that respiration data is streamed to **LabStreamingLayer (LSL)** on a local network. Streaming to LSL allows for simple synchronisation of respiration data with other physiological data (e.g. neural data). Breath acquisition modules are meant to be **swappable**. For example, a ESP microcontroller can stream directly to LSL, a python script that is connected to respiration recording hardware or a DSP script (e.g. C++) that accesses the microphone can stream.

Layer 2: Networking
For making respiration data available to applications, independent of the platform and language they are written in, samples are made available via **WebSockets**. WebSockets are network data ports that client software can connect to and read data from. A Python server reads the reparation stream from LSL and forwards the data to WebSockets.

Layer 3: Application
Applications can be created on any platform as long as WebSockets are supported. **Javascript and P5.js** are used for prototyping with 2D or simple 3D visuals. Those can later be hosted by **Electron** standalone applications so that no web browser is required. **Unity** can be used for more video game adjacent applications with more complex 3D visuals. As data is streamed over the (local) network, it is possible to have multiple, wirelessly connected clients in the experiment setup.

Possible to use mobile / distributed / multiple deviced